

Review



- Last Class:
 - Review of instruction set architecture (ISA)
 - Instruction format
 - Addressing mode
- Today's class:
 - Review of pipeline

Computer Pipelines



- RISC features that facilitates efficient pipelining:
 - all instructions have the same length,
 - registers located in the same place in instruction format,
 - memory operands only in loads or stores
- Non-pipelined RISC architecture
 - at any moment, the hardware is working on **only one instruction**
- Review – you should know about:
 - multiplexors,
 - register files,
 - ALU's
 - arithmetic Vs logical shifts
 - program counter (PC) and instruction register (IR),

Implementing the RISC architecture (Section C.3)

Arithmetic/logic instructions (R-type rd, rs1, rs2)

funct7	rs2	rs1	funct3	rd	opcode
--------	-----	-----	--------	----	--------

- 1) fetch instruction
- 2) read registers rs1 and rs2
- 3) compute result (use ALU)
- 4) write to register rd

Load instructions (lw rd, c(rs1))

Imm[11:0]	rs1	funct3	rd	opcode
-----------	-----	--------	----	--------

- 1) fetch instruction
- 2) read register rs1 (and rs2 ??)
- 3) use ALU to compute memory address
= content of rs1 + c
- 4) read from memory
- 5) write to register rd

Store instructions (sw rs2, c(rs1))

Imm[11:5]	rs2	rs1	funct3	Imm[4:0]	opcode
-----------	-----	-----	--------	----------	--------

- 1) fetch instruction
- 2) read registers rs1 and rs2
- 3) use ALU to compute memory
address = content of rs1 + c
- 4) write to memory at address stored
in rs2

Branch instructions (beq rs1, rs2, L)

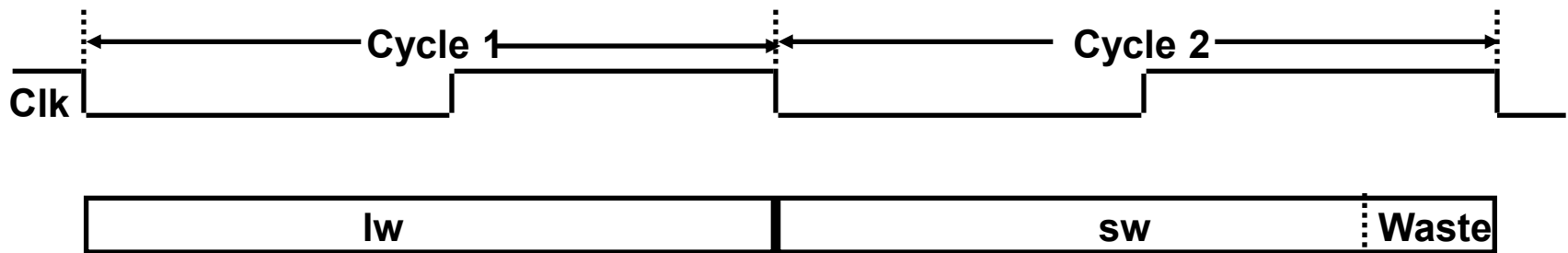
Imm[11:5]	rs2	rs1	funct3	Imm[4:0]	opcode
-----------	-----	-----	--------	----------	--------

- 1) fetch instruction
- 2) read registers rs1 and rs2
- 3) use ALU to compute branch
address = PC + offset and
evaluate branch condition (rs1 == rs2)
- 4) update the PC (if condition satisfied)

Increment the PC by 4
(in step 1)

Single Cycle Disadvantages & Advantages

- Uses the clock cycle inefficiently – the clock cycle must be timed to accommodate the **slowest** instruction
 - Load, needs address calculation



- May be wasteful of area since some functional units (e.g., adders) must be duplicated since they can not be shared during a clock cycle

but

- Single cycle datapath is simple and easy to understand

CCT Problem



- If all instructions must complete within one clock cycle, then the cycle time has to be large enough to accommodate the *slowest* instruction.
- For example, `lw $t0, -4($sp)` needs 8ns, assuming the delays shown here.

reading the instruction memory	2ns	}	8ns
reading the base register \$sp	1ns		
computing memory address \$sp-4	2ns		
reading the data memory	2ns		
storing data back to \$t0	1ns		

CCT Problem



- If we make the cycle time 8ns then *every* instruction will take 8ns, even if they don't need that much time.
- For example, the instruction `add $s4, $t1, $t2` really needs just 6ns.

reading the instruction memory	2ns	}	6ns
reading registers \$t1 and \$t2	1ns		
computing \$t1 + \$t2	2ns		
storing the result into \$s0	1ns		

Another Example

- With these same component delays, a **sw** instruction would need 7ns, and **beq** would need just 5ns.
- Let's consider the gcc instruction mix:

Instruction	Frequency
Arithmetic	48%
Loads	22%
Stores	11%
Branches	19%

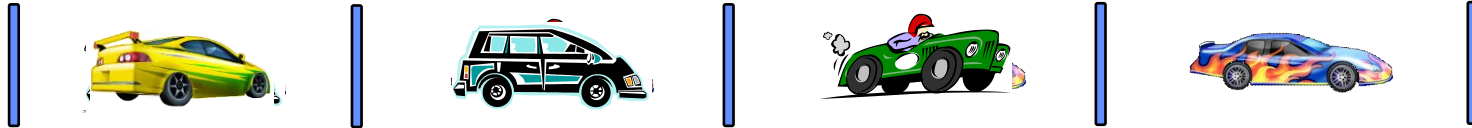
- With a single-cycle datapath, each instruction would require 8ns.
- But if we could execute instructions **as fast as possible**, the average time per instruction for gcc would be:

$$(48\% \times 6\text{ns}) + (22\% \times 8\text{ns}) + (11\% \times 7\text{ns}) + (19\% \times 5\text{ns}) = 6.36\text{ns}$$

- The single-cycle datapath is about 1.26 times slower!

Pipelining: Its Natural!

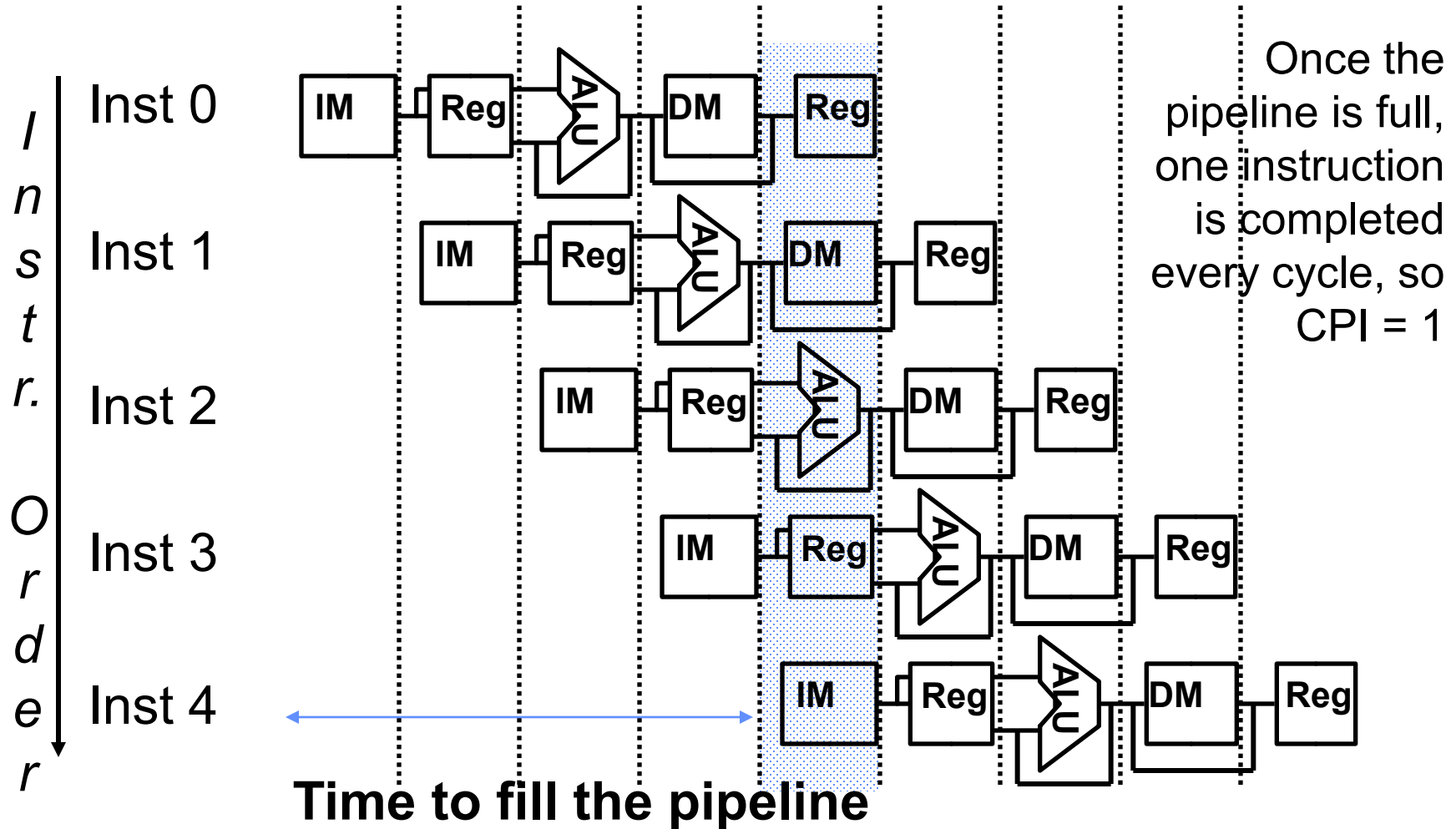
- Car wash Example



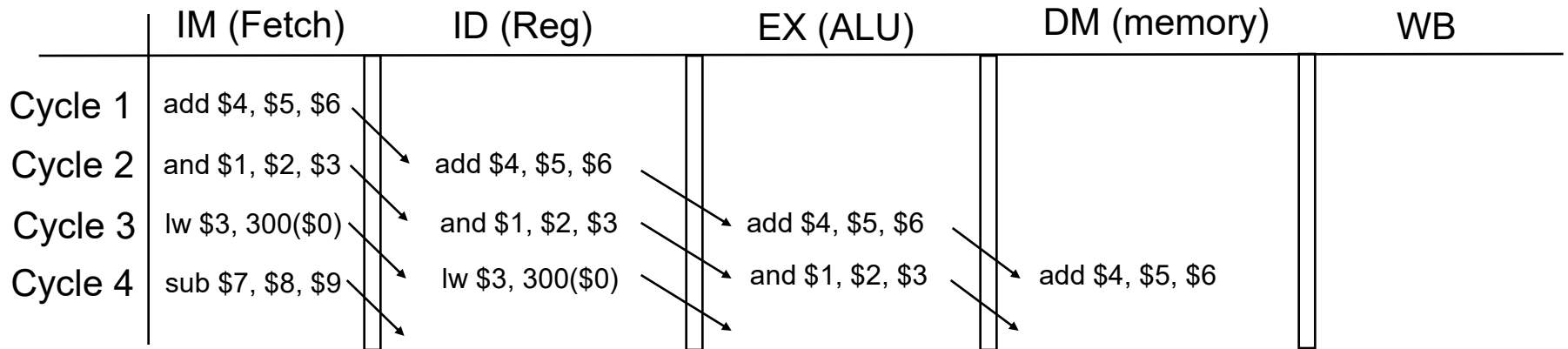
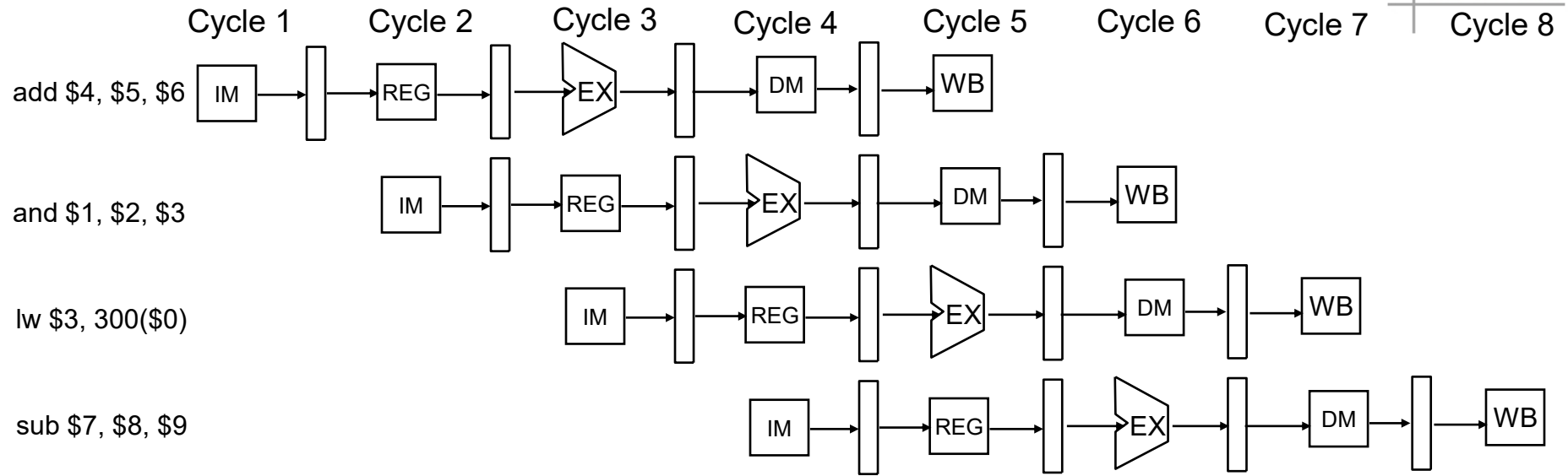
- Pipelining doesn't help **latency** of single task, it helps **throughput** of entire workload
- **Multiple** tasks operating simultaneously
- Potential speedup = **Number pipe stages**
- Pipeline rate limited by **slowest** pipeline stage
- Unbalanced lengths of pipe stages reduces speedup
- Time to “**fill**” pipeline and time to “**drain**” it reduces speedup

Why Pipeline? For Performance!

Time (clock cycles)



Visualizing Pipelining (two equivalent diagrams)



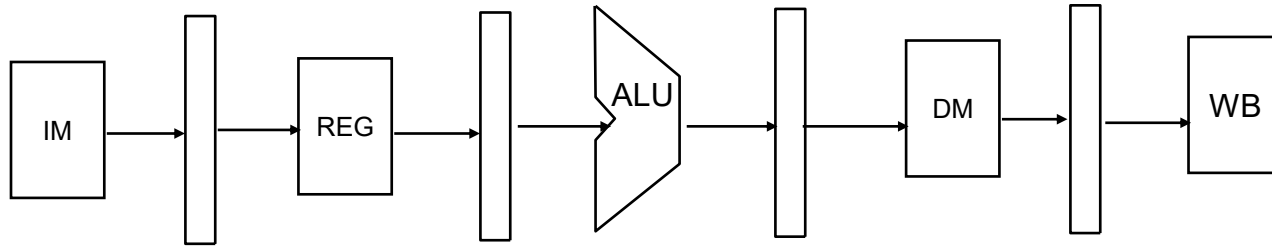
Pipelining puts additional demands on memory bandwidth

Limits to pipelining (Sections C.1 and C.2)



- **Hazards** prevent next instruction from executing during its designated clock cycle
 - **Structural hazards**: HW cannot support some combination of instructions.
 - **Data hazards**: An instruction depends on result of prior instruction still in the pipeline.
 - Dependencies backward in time cause hazards
 - Can usually resolve hazards by waiting
 - **Control hazards**: Pipelining of branches & other instructions *stall* the pipeline until the hazard *bubbles* in the pipeline
- The pipeline control logic must detect hazards,
- And take action to fix them.

Structural Hazards



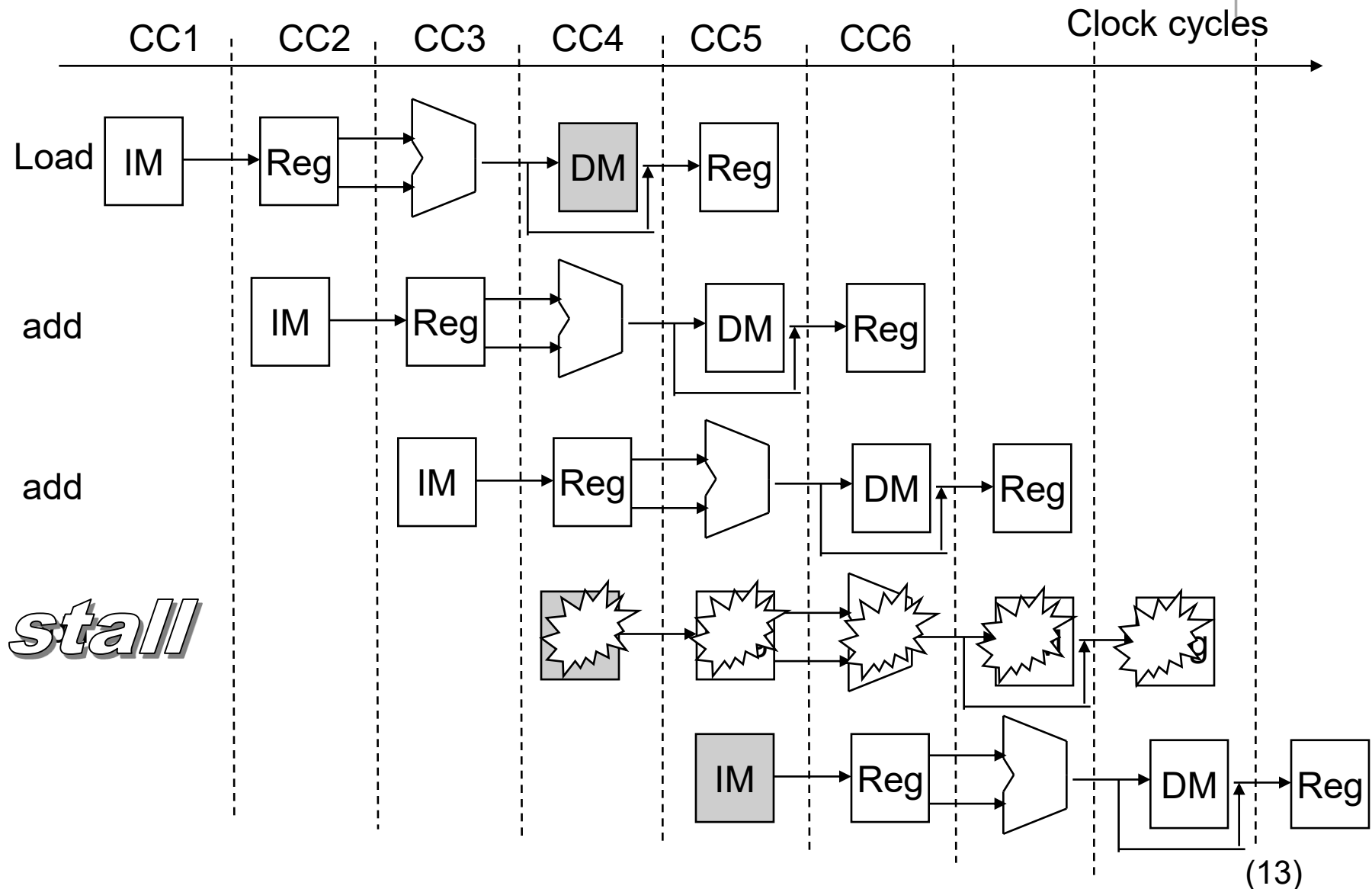
Potential problem: Both REG and WB use the register file

Solution : Read from register file during the first half of a cycle and write to register file during the second half of a cycle.

Potential problem: Both IM and DM use memory

Solution : use separate memories (caches)

Structural Hazards (assuming a single memory)



Speed Up Equation for Pipelining



$$CPI_{\text{pipelined}} = \text{Ideal CPI} + \text{stall cycles per instruction}$$

$$\text{Speedup} = \frac{CPI_{\text{unpipelined}}}{CPI_{\text{pipelined}}} \times \frac{\text{Clock Cycle}_{\text{unpipelined}}}{\text{Clock Cycle}_{\text{pipelined}}}$$

Example:

- Machine A: pipelined (with some depth) and dual ported memory
- Machine B: pipelined (same depth as A), but single ported memory, and a 1.05 times faster clock rate
- Ideal CPI = 1 for both, and lw/sw are 40% of instructions executed

$$\text{SpeedUp}_A =$$

$$\text{SpeedUp}_B =$$

$$\text{SpeedUp}_A / \text{SpeedUp}_B =$$

Three types hazards caused by Data Dependence



- **Read After Write (RAW)**

$Instr_{i+k}$ reads operand before $Instr_i$ writes it

- **Write After Read (WAR)**

$Instr_{i+k}$ writes operand *before* $Instr_i$ reads it

- Gets wrong operand
- Can't happen in RISC 5 stage pipeline (why?)

- **Write After Write (WAW)**

$Instr_{i+k}$ writes operand *before* $Instr_i$ writes it

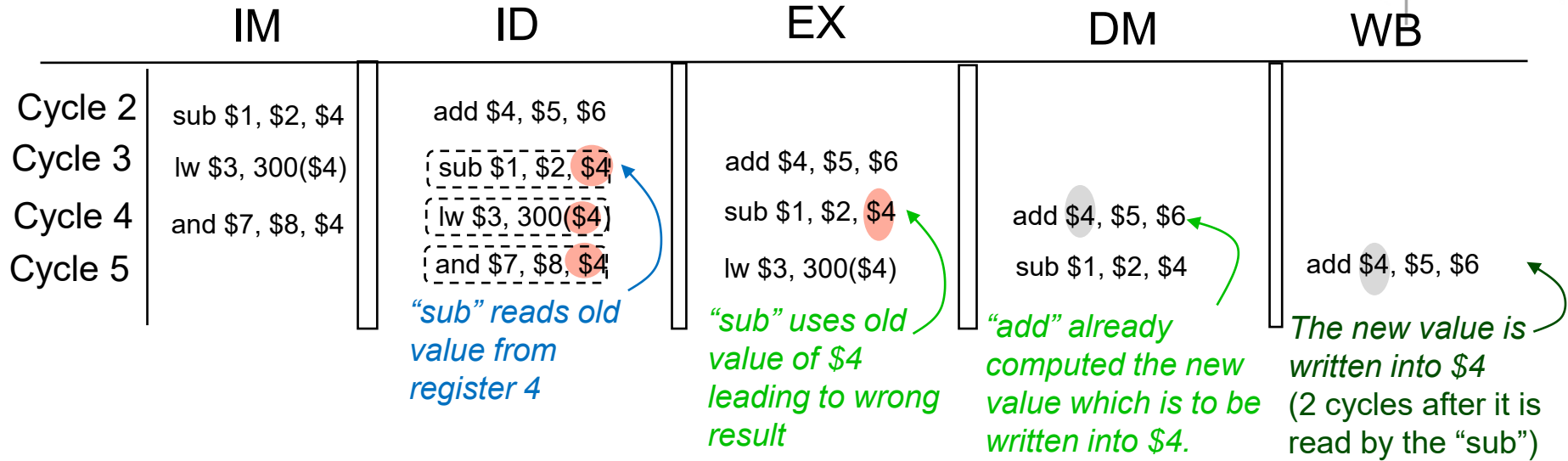
- Leaves wrong result
- Can't happen in RISC 5 stage pipeline (why?)

- Forwarding mitigates data hazards

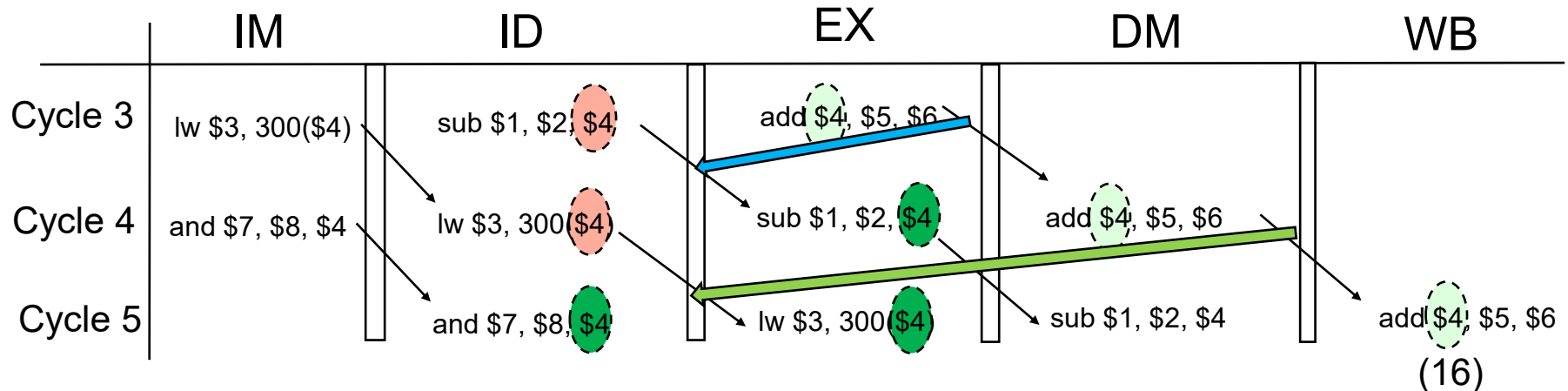
- Will see WAR and WAW in later more complicated pipes



Register-to-register data Hazards



Use forwarding to mitigate data hazards?



Data Forwarding Control Conditions



1. EX/MEM Forward Unit:

```
if (EX/MEM.RegWrite
and (EX/MEM.RegisterRd != 0)
and (EX/MEM.RegisterRd == ID/EX.RegisterRs))
    ForwardA = 10
if (EX/MEM.RegWrite
and (EX/MEM.RegisterRd != 0)
and (EX/MEM.RegisterRd == ID/EX.RegisterRt))
    ForwardB = 10
```

Forwards the result from the previous instr. to either input of the ALU

2. MEM/WB Forward Unit:

```
if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd != 0)
and (MEM/WB.RegisterRd == ID/EX.RegisterRs))
    ForwardA = 01
if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd != 0)
and (MEM/WB.RegisterRd == ID/EX.RegisterRt))
    ForwardB = 01
```

Forwards the result from the second previous instr. to either input of the ALU

Yet Another Complication!

- Another potential data hazard can occur when there is a conflict between the result of the WB stage instruction and the MEM stage instruction – which should be forwarded?

